

Process Update 4

ATLAS-Agent: A Multi-Purpose Tool-Using LLM Agent System for End-to-End Task Automation

22 July 2026

Role	Team
Group Leader	Cui Yuqing
Members	Cui Yuqing, Kuang Yang, Huang Danqi, Wang Junjie, Zhang Zhiyu
Supervisor	Dr. Loretta Choi

Update Focus

This update closes the feedback loop from the previous review: it states the project purpose as a concrete problem, identifies the unique control layer around the LLM, quantifies model selection, and reports completed ablation studies.

Teacher Feedback	Response in Update 4	Evidence
1. Clarify purpose and problem	Define the completion gap in end-to-end agent tasks	Section 1
3. Show unique highlights and an application	Two-level verification, trace, reuse, planner; Tokyo case study	Sections 2-3
6. Quantify model selection	Feature score, decision rules, five model profiles, 30-task comparison	Sections 4-5
7. Conduct ablation studies	Verification, routing, workflow reuse, and Task Planner comparisons	Sections 5-6

1. Project Purpose: Closing the Completion Gap

Purpose statement: ATLAS-Agent turns a natural-language goal into usable, structured, and verifiably complete deliverables across multiple task domains.

The problem we are solving

- A language model can generate fluent text, but real tasks also require external information, tool execution, files, dashboards, and state changes.
- A normal ReAct-style agent may stop when the model produces a final answer, even when a required file is missing or the output violates a budget, date, or schema constraint.
- Long tasks are difficult to inspect and resume, while repeated tasks often rediscover the same tool sequence and schema from scratch.
- Using one fixed model for every task wastes cost on simple requests and may underperform on coding, design, or deep reasoning tasks.

Operational definition

Stage	Typical text-first agent	ATLAS-Agent
Input	Natural-language prompt	Goal, history, tools, workspace, and task contract
Execution	Model decides when it is finished	ReAct loop with tools, plugins, MCP, Skills, HITL, and sandbox
Acceptance	Trust the final response	Verify artifacts and the final delivery contract
Evidence	Terminal logs or final prose	Structured Task Trace with failures, repair, tokens, and duration
Repeated work	Start from scratch	Reuse a successful verified workflow

Success criteria

- Cross-domain: one runtime supports travel, rental, paper reproduction, and general tasks.
- Deliverable-based: outputs include machine-readable JSON or Markdown and interactive HTML, not only prose.
- Verifiable: a task is successful only when deterministic acceptance checks pass.
- Observable and reusable: execution evidence is recorded and successful procedures can guide similar tasks.

2. What Is Unique About ATLAS-Agent?

The main contribution is not a new base model. It is the modular control layer around the model.

Highlight	What ATLAS-Agent implements	Why it matters
Two-level Verification	Plugin validators check artifact content; TaskVerifier checks the complete delivery contract before success	Separates the model's belief from system evidence
Structured Task Trace	Records task context, routing, tool calls, failures, repair, verification, tokens, and duration	Makes debugging, evaluation, and cost claims auditable
Workflow Reuse	Extracts a minimal tool path only from successful verified traces and matches it to similar tasks	Reduces repeated exploration and tool calls
Task Planner	Maintains explicit steps, status, persisted state, and interruption recovery	Makes long-horizon execution visible and resumable
Five-model Routing	Uses bilingual feature extraction, difficulty scoring, and mutually exclusive rules	Matches task type to model capability and reasoning budget
Domain Plugins	Travel, rental, and paper pipelines generate structured data plus interactive HTML	Demonstrates multi-purpose delivery through one Tool interface

How the modules reinforce one another

Plugin artifact -> domain validation -> structured repair if needed -> final task contract -> successful Task Trace -> reusable Workflow

- The LLM remains responsible for flexible reasoning, tool choice, arguments, and user-facing explanation.
- The runtime owns deterministic responsibilities: workspace boundaries, maximum steps, approval, persistence, verification, and final status.
- This division makes critical behavior testable without requiring the model to judge its own success.

3. Application Evidence: Why Verification Is Necessary

Case study: an intentionally invalid 3-day Tokyo travel plan

Step	Observed output	System response
1. Generate	The prompt forces only 2 itinerary days and a budget above HKD 10,000	Artifact is created, so file existence alone is not enough
2. Detect	DAY_COUNT_MISMATCH and BUDGET_OVERFLOW	Plugin validator returns error code, affected field, evidence, and repair hint
3. Repair	Day 3 is added and the budget is reduced to exactly HKD 10,000	The repaired artifact passes the domain rules
4. Accept	Required itinerary and dashboard artifacts are present	Final TaskVerifier checks the complete delivery contract

2	constraint violations detected
2	violations repaired
HKD 10,000	final budget
3 days	final itinerary

Why the feature is important

- A plausible-looking answer can still be operationally wrong. Deterministic checks catch date continuity, time overlap, required fields, and numerical constraints.
- Structured repair is more targeted than asking the model to try again without evidence.
- The same validation adapter is reused across travel, rental, and paper reproduction, while each plugin supplies its own domain rules.

Control experiment: false completion with missing files

Domain	Missing artifact	Verification failures	Repair attempts	Final status
Travel	trip_plan.json	2	1	Rejected
Rental	rental_listings.json and rental_scores.json	2	1	Rejected
Paper	repro_checklist.md	2	1	Rejected

Without final Task Verification, the baseline ReAct control flow would accept a text response as success when no tool call is present. With verification enabled, all three false completion attempts were rejected.

4. Quantified Model Selection for a Multi-Purpose Agent

Routing principle: recommend a model profile from observable task features and difficulty, rather than claiming one model is best for every task.

Feature-based difficulty score

Feature	Score	Detection intent
Coding	+2.0	Implementation, debugging, repository or code keywords
Reasoning	+2.5	Argumentation, proof, deep comparison, theoretical analysis
Analytical	+1.5	Data analysis, statistics, quantitative evaluation
Design	+1.0	Architecture, planning, system design
Complex	+1.5	Multiple features, dependencies, or chain-like tasks
Retrieval	-1.0	Pure query or information lookup
Length bonus	+0.5 / +1.0 / +2.0	More than 150 / 300 / 500 characters

Mutually exclusive recommendation rules

Priority	Condition	Profile	Provider/model in experiment
1	Pure coding and complex or difficulty >= 6	CodeMaster	Claude Opus 4.6
2	Design and complex and difficulty >= 6	PlanExpert	GPT-5.4
3	Pure reasoning and difficulty >= 7	ReasoningPro	Gemini 3.1 Pro
4	Other high difficulty >= 7	ReasoningPro	Gemini 3.1 Pro
5	Difficulty in [4.0, 7.0)	Balanced	DeepSeek V4 Pro
6	Difficulty < 4.0	UltraFast	DeepSeek V4 Flash

Why this fits a multi-purpose target

- The criteria use task properties, not domain names, so they apply to travel, rental, research, coding, design, and general requests.
- The five profiles separate fast retrieval, balanced general work, coding, architecture planning, and deep reasoning.
- Rules are explainable: the terminal shows detected features, difficulty score, and the recommended model.

5. Routing Ablation: Fixed Model vs. Five-Model Routing

Experimental design

- 30 labelled tasks: 8 Simple, 7 Balanced, 7 Coding, 4 Design, and 4 Reasoning.
- Baseline: every task uses the Balanced model. Enhanced condition: the complete routing system selects among five profiles and applies task-specific prompt enhancement.
- Metrics: quality, average cost, tokens, value for money, model-selection accuracy, and result stability.

Metric	No Routing	With Routing	Change
Quality score	73.88	88.25	+19.4%
Average cost	\$0.583	\$0.467	-19.9%
Average tokens	428	365	-14.7%
Value for money	1.80	2.25	+25.0%
Model-selection accuracy	N/A	100%	Perfect on labelled set
Standard deviation	5.2	1.8	-69.2%

+19.4%	quality
-19.9%	average cost
-14.7%	tokens
-69.2%	score variation

Interpretation

- The gain is not only a cost reduction: quality and stability improve at the same time.
- The result supports the teacher's suggested approach: recommend a provider/model area for a defined task type instead of forcing a universal model.
- The 100% selection accuracy applies to this labelled 30-task set and should not be treated as universal accuracy for ambiguous or mixed-intent prompts.

Ablation conclusion: task-aware routing outperforms a fixed Balanced model on quality, efficiency, and stability under the defined benchmark.

6. Ablation Suite for Reliability and Efficiency

Module	Baseline	Enhanced condition	Observed result
Final Verification	Accept final text when no tool call remains	Check required artifacts; one structured repair round	3/3 missing-artifact tasks rejected; none stored as successful workflow
Workflow Reuse	First execution without workflow guidance	Verified minimal tool path injected for similar tasks	9/9 final contracts passed; 6/6 reuse runs succeeded
Task Planner	Original ReAct state only	Explicit steps, status, persistence, recovery	Shows progress, avoids repeating completed steps, resumes after interruption
Routing	All tasks use Balanced	Five-model task-aware routing	Quality +19.4%; cost -19.9%; tokens -14.7%

Workflow Reuse: domain-level efficiency

Domain	Baseline tokens	Mean reuse tokens	Token change	Tool-call change	Verification
Travel	140,382	111,723	-20.4%	-12.5%	3/3 passed
Rental	145,331	84,209	-42.1%	-33.3%	3/3 passed
Paper	65,953	17,773	-73.0%	-62.5%	3/3 passed

Task Planner qualitative ablation

Method	Completes normal task	Shows progress	Repeats completed steps after interruption	Resumes
Original ReAct	Yes	No	Yes	No
ReAct + Task Planner	Yes	Yes	No	Yes

What the ablations demonstrate

- Verification contributes a measurable acceptance boundary, rather than only improving the wording of the final answer.
- Workflow Reuse preserves final acceptance while reducing exploration overhead; the benefit varies with task structure.
- Task Planner contributes recoverability and visibility even when both variants can complete an uninterrupted task.

7. Current Status, Limitations, and Next Steps

Completed system scope

Area	Completed evidence
Core runtime	Asynchronous ReAct loop, multi-provider clients, CLI, sessions, context summarisation
Capabilities	File, shell, web, Python sandbox, MCP, progressively loaded Skills, domain plugins
Safety and reliability	Workspace confinement, HITL, intent-aware controls, two-level Verification, structured repair
Observability and efficiency	Task Trace, Workflow Reuse, Task Planner, five-model Routing
Applications	Travel, rental, and paper reproduction with JSON/Markdown and interactive HTML
Engineering quality	35 test_*.py files plus integration demonstrations and retained trace records

Limitations and responsible interpretation

- Workflow evaluation contains one baseline and two reuse tasks per domain; larger repeated experiments are required for population-level claims.
- The Routing benchmark contains 30 labelled tasks with a fixed distribution; ambiguous and mixed-intent prompts remain future work.
- Deterministic validation proves observable structure and constraints, but does not by itself prove factual freshness or subjective usefulness.
- Task Planner evidence is qualitative and prototype-oriented; saved tokens and time after recovery have not yet been measured at scale.

Next steps

- Run 5-10 independent baseline/reuse pairs per domain with fixed models, timestamps, and confidence intervals.
- Replace keyword-only workflow matching with hybrid keyword and embedding retrieval plus compatibility checks.
- Separate structural verification from source freshness checks for weather, listings, links, and repository versions.
- Integrate Task Planner state transitions with Trace and revalidate completed artifacts on resume.

Individual contributions

Member	Primary contribution
Cui Yuqing (Leader)	Core Agent architecture, ReAct runtime, model APIs, tools, sandbox, HITL, sessions, integration
Kuang Yang	Task Trace, two-level Verification, Workflow Reuse, failure injection and reuse ablations
Huang Danqi	Task Planner, persistent step state, progress display, interruption recovery comparison
Wang Junjie	Plugin architecture, travel/rental/paper pipelines, validators, structured and visual deliverables
Zhang Zhiyu	Five-model Routing, scoring rules, prompt enhancement, and 30-task quantitative comparison

Sources: ATLAS-Agent Final Project Report, Sections 1, 4, 6-10, 12, and 15; Final Defense Deck, Slides 4-42; retained Task Trace and experiment records.